Trabajo 3 · Transfer learning, fine-tuning e interpretabilidad aplicados al TFM

Detección de anomalías territoriales en datos abiertos gubernamentales del Ecuador (TFM Tema 50)

Wilson Paredes · Diana Tandalla · Alexander Luna · Bryan Sarmiento — octubre de 2026 · Notebook: Trabajo3_TransferLearning_TFM_Tema50.ipynb

1. Tarea, datos y modelo de la entrega anterior

El TFM consolida 24 instituciones del portal de datos abiertos y detecta combinaciones cantón-año atípicas con un consenso de tres detectores no supervisados (Isolation Forest, LOF y One-Class SVM). La tarea de este trabajo es su **modelo sustituto supervisado**: clasificar si un cantón-año es anomalía (al menos 2 de 3 votos), para puntuar casos nuevos sin reajustar el ensamble y explicar qué indicador pesa en cada uno. Los datos son la matriz efectiva del capítulo 4: 807 filas (218 cantones, 2021–2025) × 10 indicadores de 8 instituciones, expresados como logaritmo del cociente de localización (tasa por mil habitantes del cantón ÷ tasa nacional del año). La clase positiva es ≈4–5 % de las filas. Las etiquetas se calculan **sin fuga de información**: los detectores se ajustan solo con el entrenamiento y se aplican a la prueba.

El modelo de la entrega anterior, entrenado desde cero, es un MLP de scikit-learn con dos capas ocultas (16 y 8 neuronas, ReLU, Adam, 150 épocas) y sobremuestreo de la clase minoritaria; se reentrena aquí con el mismo protocolo sobre la matriz actual para comparar en igualdad de condiciones, junto con una regresión logística de referencia.

2. Selección y justificación del modelo pre-entrenado

Se eligió **TabPFN v1** (Hollmann et al., 2023), una *Prior-Data Fitted Network*. Los modelos de Keras Applications o KerasNLP están pre-entrenados con imágenes o texto y no transfieren a una tabla de 10 variables numéricas; TabPFN es el modelo pre-entrenado de referencia para **clasificación tabular pequeña**, con límites (1.024 filas de entrenamiento, 100 variables, 10 clases) que contienen el caso del TFM (≈650 filas, 10 variables, 2 clases). Fue pre-entrenado con millones de tablas sintéticas generadas por un *prior* de modelos causales estructurales y redes bayesianas aleatorias, de modo que aprendió a aproximar la inferencia bayesiana para clasificación (Müller et al., 2022): recibe el entrenamiento como contexto y clasifica las filas nuevas en un solo paso hacia adelante, sin descenso de gradiente. Es una ventaja directa para el TFM, donde hay pocas anomalías etiquetadas y los métodos profundos entrenados desde cero suelen rendir peor que en tablas grandes (Grinsztajn et al., 2022).

Tabla 1. Arquitectura de TabPFN v1 y cambios aplicados.

Componente	TabPFN v1 original	Uso en este trabajo
Codificador de variables	Lineal 100 → 512	Congelado; las 10 variables se rellenan con ceros hasta 100
Codificador de etiqueta	Lineal 1 → 512	Congelado
Transformer	12 capas, 4 cabezas, MLP 1.024	Capas 1–10 congeladas; capas 11–12 reentrenadas
Decodificador	Lineal 512 → 1.024 → 10 clases	Reentrenado; solo se usan las salidas 0 y 1
Parámetros	25,8 M	4,7 M entrenables (18%)

Nota. Pesos oficiales del checkpoint publicado (época 42, 103 MB), descargados del repositorio automl/TabPFN, etiqueta v1.0.0.

3. Preparación del dataset y ajuste del modelo

Preparación. (i) Los faltantes (38 % de las celdas, por indicadores que no se publican todos los años) se imputan con la mediana de los demás años del mismo cantón, con estadísticos ajustados solo en entrenamiento, porque TabPFN v1 no fue pre-entrenado con muchos faltantes. (ii) Se aplica el preprocesamiento que espera el modelo: estandarización con el contexto, recorte a ±4 desviaciones, escalado por el número de variables usadas y relleno con ceros hasta las 100 entradas del codificador. (iii) Las etiquetas 0/1 ocupan las dos primeras de las 10 salidas.

Transfer learning (pesos congelados). El modelo se usa tal como fue pre-entrenado, con aprendizaje en contexto: el entrenamiento completo es el contexto y se promedia un ensamble de 8 configuraciones (permutaciones de variables y clases), como en la API oficial.

Fine-tuning. Se congelan el codificador y las 10 primeras capas, que contienen el conocimiento general sobre tablas, y se reentrenan las 2 últimas capas y el decodificador. Se entrena por episodios, igual que en el pre-entrenamiento: en cada paso el entrenamiento se divide en contexto (70 %) y consulta (30 %, estratificada), y la pérdida se calcula sobre la consulta. Se usa entropía cruzada ponderada por la raíz del desbalance, Adam con tasa 1e-5, 60 pasos y recorte del gradiente a 1. La pérdida de la consulta bajó de 0,237 a 0,144 (media de los 10 primeros y 10 últimos pasos). La

partición de prueba nunca interviene. Un control con el modelo original y el mismo preprocesamiento simple aísla el efecto del fine-tuning.

4. Evaluación y comparación con el modelo desde cero

Se repite el protocolo de la entrega anterior: tres esquemas de división (filas al azar; cantones nuevos, que reserva el 20 % de los cantones con todos sus años; provincias nuevas) y 10 repeticiones, de las que quedan 28 folds válidos por modelo al descartar los que tienen menos de 3 anomalías en prueba. Con ≈5 % de positivos, la exactitud no informa; se reportan F1 y *recall* de la clase anomalía (umbral 0,5) y PR-AUC, independiente del umbral (Tabla 2 y Figura 1).

Tabla 2. Desempeño en la clase anomalía (media ± desviación estándar entre folds).

Esquema	Modelo	Precisión	Recall	F1	PR-AUC	s/fold
Filas al azar	MLP desde cero	0,36 ± 0,18	0,36 ± 0,27	0,34 ± 0,20	0,36 ± 0,22	0,8
	Regresión logística	0,09 ± 0,06	0,52 ± 0,33	0,15 ± 0,10	0,25 ± 0,16	0,0
	TabPFN transfer	0,63 ± 0,40	0,32 ± 0,22	0,42 ± 0,28	0,66 ± 0,15	3,5
	▲ TabPFN fine-tuning	0,55 ± 0,26	0,64 ± 0,23	0,56 ± 0,19	0,70 ± 0,19	23,7
Cantones nuevos	MLP desde cero	0,56 ± 0,34	0,17 ± 0,11	0,25 ± 0,16	0,46 ± 0,15	0,7
	Regresión logística	0,18 ± 0,06	0,49 ± 0,22	0,26 ± 0,09	0,26 ± 0,12	0,0
	TabPFN transfer	0,89 ± 0,33	0,35 ± 0,18	0,50 ± 0,23	0,81 ± 0,19	3,4
	▲ TabPFN fine-tuning	0,79 ± 0,20	0,72 ± 0,21	0,75 ± 0,18	0,88 ± 0,14	23,8
Provincias nuevas	MLP desde cero	0,43 ± 0,35	0,26 ± 0,28	0,27 ± 0,21	0,37 ± 0,23	0,7
	Regresión logística	0,13 ± 0,05	0,50 ± 0,25	0,20 ± 0,07	0,20 ± 0,10	0,0
	TabPFN transfer	0,71 ± 0,35	0,26 ± 0,18	0,37 ± 0,23	0,73 ± 0,17	3,4
	▲ TabPFN fine-tuning	0,67 ± 0,19	0,73 ± 0,19	0,66 ± 0,10	0,80 ± 0,12	23,8

Nota. Tiempo en CPU por fold (ajuste y predicción). El fine-tuning incluye 60 pasos de entrenamiento.

Figura 1. F1 y PR-AUC por esquema de división; las barras de error son una desviación estándar entre folds.

Resultados. TabPFN con fine-tuning supera al MLP desde cero en los tres esquemas y en ambas métricas, con diferencias pareadas cuyo intervalo de confianza del 95 % (bootstrap) no incluye el cero. En cantones nuevos, el escenario que más interesa al TFM, el F1 sube de 0,25 a 0,75 (diferencia 0,49, IC [0,41; 0,57]) y la PR-AUC de 0,46 a 0,88. Sin ajuste, el modelo pre-entrenado ya ordena bien los casos (PR-AUC de 0,81 con pesos congelados), pero con el umbral 0,5 es conservador: su precisión es alta (0,89) y su recall bajo (0,35). El fine-tuning con pérdida ponderada desplaza esa frontera y duplica el recall (0,72) sin perder mucha precisión. El control con el mismo preprocesamiento (PR-AUC 0,81 en cantones nuevos) confirma que la mejora se debe al ajuste de pesos y no al preprocesamiento.

Comparación con la entrega anterior. Con la matriz anterior, de 4 indicadores y 1.284 filas, el mismo MLP alcanzaba F1 0,63 en cantones nuevos; con la matriz actual, de 10 indicadores y menos filas positivas por variable, cae a 0,25. TabPFN no sufre esa caída porque su capacidad de generalizar viene del pre-entrenamiento y no de los pocos ejemplos del TFM. **Ventajas:** mejor desempeño y generalización a territorios no vistos, sin búsqueda de hiperparámetros, y el modelo con pesos congelados no requiere entrenamiento. **Desventajas:** el fine-tuning cuesta ≈25 s por fold frente a <1 s del MLP; el modelo pesa 26 M de parámetros y depende de un checkpoint externo; está limitado a 1.024 filas de contexto, por lo que no escalaría al nivel parroquial sin submuestreo, y es menos transparente, lo que hace necesaria la explicabilidad.

5. Explicabilidad con SHAP

Elección. TabPFN no expone importancias propias y Grad-CAM es para imágenes. Se usa SHAP (Lundberg y Lee, 2017) porque es agnóstico al modelo, tiene base en los valores de Shapley y da una contribución por indicador para cada cantón-año, que es lo que necesita la revisión experta del TFM. **Implementación:** Kernel SHAP sobre la probabilidad de anomalía del modelo con fine-tuning, en la división por cantones nuevos, con 20 filas de fondo (k-medias del entrenamiento) y 300 evaluaciones por fila, para las 24 anomalías de prueba y 30 filas normales.

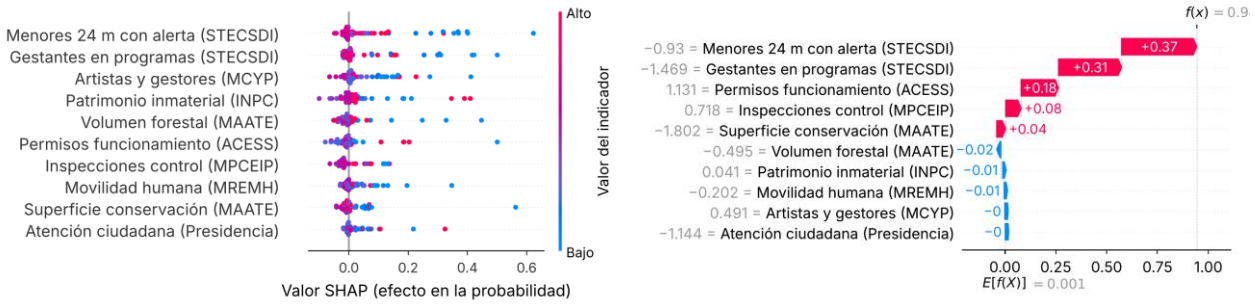


Figura 2. Izquierda: contribución de cada indicador a la probabilidad de anomalía (cada punto es un cantón-año; color: valor del indicador). Derecha: explicación del caso con mayor probabilidad, Rumiñahui (Pichincha) 2022.

Qué revela. (1) Los dos indicadores de la Secretaría Técnica Ecuador Crece sin Desnutrición Infantil son los de mayor importancia global (media |SHAP| de 0,092 y 0,069), seguidos del registro de artistas y gestores culturales (MCYP) y del patrimonio cultural inmaterial (INPC). (2) La relación es mayoritariamente de **valores bajos que empujan hacia anomalía** (puntos azules a la derecha): el modelo marca coberturas o tasas inusualmente bajas, coherente con lo que el capítulo 4 identifica como posibles coberturas parciales de los programas piloto. (3) En el 71% de las anomalías, el indicador con mayor |SHAP| coincide con el más desviado (máximo |z|) que el TFM usa para explicar cada caso: SHAP confirma la explicación sencilla en la mayoría de los casos y la corrige en el resto, donde pesa una combinación de indicadores. (4) El caso de la derecha es el hallazgo más útil: la probabilidad de 0,94 se debe sobre todo a Menores 24 m con alerta (STECSDI), Gestantes en programas (STECSDI) y Permisos funcionamiento (ACESS), que en ese año **no fueron publicados y están imputados** con la historia del cantón. En el 25% de las 24 anomalías explicadas, la mayor contribución viene de un valor imputado. SHAP hace visible un artefacto de imputación que la métrica no muestra y respalda la decisión del TFM de exigir evidencia observada para que un caso entre en la lista que se entrega al decisor.

6. Conclusiones

El transfer learning desde un modelo pre-entrenado para tablas fue más eficaz que entrenar desde cero en un problema con pocas anomalías etiquetadas: con pesos congelados ya ordena mejor los casos, y el fine-tuning de las dos últimas capas y el decodificador eleva el F1 en cantones nuevos de 0,25 a 0,75. SHAP mostró qué indicadores sostienen cada alerta y descubrió que muchas dependen de valores imputados, lo que convierte la explicabilidad en un control de calidad del propio *pipeline* del TFM. Queda como límite que el modelo sustituto reproduce el consenso de los detectores, no la verdad de terreno: su utilidad final dependerá de la revisión experta prevista en el TFM.

7. Referencias

Grinsztajn, L., Oyallon, E. y Varoquaux, G. (2022). Why do tree-based models still outperform deep learning on typical tabular data? *Advances in Neural Information Processing Systems, 35* (Datasets and Benchmarks).

Hollmann, N., Müller, S., Eggensperger, K. y Hutter, F. (2023). TabPFN: A transformer that solves small tabular classification problems in a second. *International Conference on Learning Representations (ICLR 2023)*.

Lundberg, S. M. y Lee, S.-I. (2017). A unified approach to interpreting model predictions. *Advances in Neural Information Processing Systems, 30*, 4765–4774.

Müller, S., Hollmann, N., Pineda Arango, S., Grabocka, J. y Hutter, F. (2022). Transformers can do Bayesian inference. *International Conference on Learning Representations (ICLR 2022)*.

8. Enlace Google Colab desde GitHub

https://github.com/Alexander-Luna/NeuralNetworks-DeepLearning/blob/main/SEMANA%203/Trabajo3_TransferLearning_TFM_Tema50.ipynb